

ユニット 8・第13-14週

ネットワーク：ソケット

📅 第13-14週・日程は未定 </> 演習 TD7（解答 + 生成AIチャレンジ） ~ アニメーション 2 本

演習 TD7・解答付き

ソケットによるネットワーク：最小のサーバから 5 つのモデルを持つ Web サーバまで

ソケットはファイルと同じように使う（`read`、`write`、`close`）が、別々のマシンのプロセスをつなぐ。この演習はすべてを組み合わせる：クライアントごとの `fork`、接続ごとのスレッド、そして多くのクライアントを同時に処理するための TD6 の生産者-消費者。

☑ 課題：実習レポート

この実習は短い LaTeX レポート（必要なことだけ）で提出します。扱う内容、テンプレート、例、Prism のガイドは「課題」タブにあります。生成 AI は申告すれば利用できます。

演習問題

- | | | | |
|---|---------------------------------|---|--|
| 1 | サーバの最低限 | 3 | Web サーバ：反復型、接続ごとのスレッド、プール |
| 2 | エコーサーバ：反復型と <code>fork</code> 版 | 4 | 探究：TCP の状態、 <code>listen</code> のキュー、負荷の下モデル、UDP |

TD7 ・ 1

● 解答済み・テスト済み

サーバの最低限

課題。接続を 1 つ待つて終了するプログラム（`telnet` か `nc` で試す）：`socket`（IPv4、接続型）→ `INADDR_ANY` とネットワークバイト順のポート（`htons`）を入れた `sockaddr_in` → `bind` → `listen` → `accept` → `close`。

解答

```
C tds/td7/sock.c
```

```
/*
```

```
 * TD7 - exercise 1: the server side of a TCP connection, step by step.
```

```

*  ./sock [port]      (default 7000)  then, from another terminal:
telnet localhost 7000
*  socket → bind → listen → accept → close.
*/
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000

int main(int argc, char *argv[])
{
    int port = argc > 1 ? atoi(argv[1]) : PORT;

    /* 1. An IPv4 (AF_INET), connected, reliable byte stream (SOCK_STREAM =
    TCP). */
    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");

    /* Allow restarting the server right away (otherwise bind fails for ~60
    s: TIME_WAIT). */
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
    "setsockopt");

    /* 2. Local address: any interface of this machine, the chosen port
    (network byte order). */
    struct sockaddr_in addr = { 0 };
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);

    /* 3. and 4. Bind the address, start listening (queue of 1 pending
    connection). */
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 1) == -1, "listen");
    printf("listening on port %d\n", port);
    fflush(stdout);

    /* 5. accept blocks until a client connects and returns a NEW descriptor
    for it. */

```

```
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");

/* 6. Close the connection and the listening socket. */
exit_if(close(client) == -1 || close(s) == -1, "close");
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./sock 37634 & sleep 0.3; nc -z 127.0.0.1 37634 && echo 'client connected';
wait
listening on port 37634
connection accepted, closing
Connection to 127.0.0.1 37634 port [tcp/*] succeeded!
client connected
```

- `accept` はそのクライアント用の新しいディスクリプタを返す。待ち受けソケットは他のクライアントを受け付けるために残る。
- `htons` / `htonl` : ネットワークはビッグエンディアン、x86 はリトルエンディアン (TD1 で `od` を使って見た)。
- `SO_REUSEADDR` があればサーバをすぐ再起動できる：なければ、以前の接続が `TIME_WAIT` にある間 `bind` は `EADDRINUSE` で失敗する。

アニメーション A8.1・接続のライフサイクル →

TD7 ・ 2

● 解答済み・テスト済み

エコーサーバ：反復型と `fork` 版

課題。エコーサーバは受け取ったものをすべてクライアントに送り返す。ただし `quit` は接続を閉じ、次の接続を待つ。クライアントを同時にいくつか起動すると何が起きるか？ `fork` を使って複数を処理できるようサーバを変えよう。

解答



C tds/td7/echo_server.c

```

/*
 * TD7 - exercise 2: echo server. Sends back every line it receives; "quit"
 closes the
 * connection and the server waits for the next client.
 * ./echo_server [port] [-f]      -f: one child process per client (fork)
 * Without -f the server is ITERATIVE: a second telnet waits until the first
 one leaves
 * (its connection stays in the listen queue). With -f each client gets its
 own process.
 */
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000
#define LINE 1024

/* Echo line by line until end of file or "quit". Returns when the client is
done. */
static void serve(int c)
{
    char line[LINE];
    size_t len = 0;

    for (;;) {
        ssize_t n = read(c, line + len, 1);      /* byte by byte: simple,
fine for a lab */
        if (n == -1 && errno == EINTR)
            continue;
        if (n <= 0)                                /* 0: the client
closed; -1: error */
            return;
        if (line[len] == '\n' || len == LINE - 2) {
            len++;
            if (strncmp(line, "quit", 4) == 0)
                return;
        }
    }
}

```

```

        if (write_all(c, line, len) == -1)
            return;
        len = 0;
    } else {
        len++;
    }
}

}

int main(int argc, char *argv[])
{
    int port = PORT, use_fork = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) use_fork = 1;
        else port = atoi(argv[i]);
    }

    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htonl((uint16_t)port),
                                .sin_addr.s_addr = htonl(INADDR_ANY) };
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 8) == -1, "listen");
    printf("echo server on port %d (%s)\n", port, use_fork ? "one process per
client" : "iterative");
    fflush(stdout);

    /* Children that finish must not stay zombies: ignoring SIGCHLD makes the
kernel reap them. */
    if (use_fork)
        exit_if(signal(SIGCHLD, SIG_IGN) == SIG_ERR, "signal");

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c == -1 && errno == EINTR)
            continue;
        exit_if(c == -1, "accept");

        if (!use_fork) {
            serve(c);
            exit_if(close(c) == -1, "close");
        }
    }
}

```

```
        continue;
    }
    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        exit_if(close(s) == -1, "close");           /* the child does not
accept */
        serve(c);
        _exit(EXIT_SUCCESS);
    }
    exit_if(close(c) == -1, "close");           /* the parent does not
talk to the client */
    }
}
```

実際の出力

```
$ python3 tcp_client.py echo 22751 'hola\nmundo\nquit'
sent: hola      received: hola
sent: mundo     received: mundo
```

実際の出力

```
$ python3 tcp_client.py blocked 22751      # iterative server
client B got nothing after 1 s: the server is busy with client A
```

実際の出力

```
$ python3 tcp_client.py blocked 34885      # server with fork
client B got 'hi\n' after 0.00 s
```

反復型サーバは一度に1つのクライアントを処理する：クライアント A が接続している間、B は `listen` のキューで待ち、応答を受けない（テストが測る：1秒間何もなし）。`-f` では接続ごとにプロセスができ、B はすぐにエコーを受け取る。親はクライアントのソケットを、子は待ち受けソケットを閉じる。`SIGCHLD` を無視すればゾンビを避けられる。引き継いだ版は終端のないバッファに `strstr` を使って `quit` を検出していたが、今は1行ずつ読む。

Web サーバ：反復型、接続ごとのスレッド、8 スレッドのプール

課題。クライアント（ブラウザ）は GET ファイル を送り、サーバは HTTP/1.0 200 OK、MIME タイプ、内容で答える。存在とサイズを知るには stat、マップするには mmap、送るには write を使う。続いて：(1) 接続ごとに新しいスレッド。(2) 起動時に作った 8 つのスレッドが接続を待つ（サーバが接続を生産し、スレッドが消費する）。

解答

C tds/td7/http_server.c

```
/* Handles one request on the connected socket c. Never exits the process. */
static void handle(int c)
{
    char req[REQUEST], path[1024], full[2048];
    ssize_t n = read(c, req, sizeof req - 1);
    if (n ≤ 0)
        return;
    req[n] = '\0';

    if (sscanf(req, "GET %1023s", path) ≠ 1) {
        reply_error(c, "400 Bad Request");
        return;
    }
    if (strstr(path, "..") ≠ NULL) {
        /* no escape from root */
        reply_error(c, "403 Forbidden");
        return;
    }
    if (strcmp(path, "/") = 0)
        strcpy(path, "/index.html");
    snprintf(full, sizeof full, "%s%s", root, path);

    struct stat st;
    if (stat(full, &st) = -1 || !S_ISREG(st.st_mode)) {
        reply_error(c, "404 Not Found");
        /* PG204 answered
        "201" by mistake */
        return;
    }
    int fd = open(full, O_RDONLY);
    if (fd = -1) {
        reply_error(c, "403 Forbidden");
    }
}
```

```

    return;
}
char head[256];
int hn = snprintf(head, sizeof head,
                  "HTTP/1.0 200 OK\r\nContent-Type: %s\r\nContent-Length:
%lld\r\n\r\n",
                  mime_type(full), (long long)st.st_size);
if (write_all(c, head, (size_t)hn) == -1) {
    close(fd);
    return;
}
if (st.st_size > 0) {
    /* mmap of 0 bytes is
    an error */
    void *data = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE,
fd, 0);
    if (data != MAP_FAILED) {
        (void)write_all(c, data, (size_t)st.st_size); /* loops over
partial writes */
        munmap(data, (size_t)st.st_size);
    }
}
close(fd);
}

/* ----- thread per connection ----- */
static void *connection_thread(void *arg)
{
    int c = (int)(intptr_t)arg;
    handle(c);
    close(c);
    return NULL;
}

/* ----- pool: bounded queue of connected sockets ----- */
static int queue[QUEUE];
static int q_in = 0, q_out = 0;
static sem_t q_free, q_used;
static pthread_mutex_t q_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg)
{
    (void)arg;
    for (;;) {
        sem_wait(&q_used);
        pthread_mutex_lock(&q_lock);
        /* wait for a connection */

```



```

    int c = queue[q_out];
    q_out = (q_out + 1) % QUEUE;
    pthread_mutex_unlock(&q_lock);
    sem_post(&q_free);
    handle(c);
    close(c);
}
return NULL;
}

```

実際の出力

```

$ curl -s -i http://localhost:PORT/ | tr -d '\r' # mode pool
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 20

<h1>FS0 ・ UTI</h1>

```

実際の出力

```

$ curl -s -o /dev/null -w '%{http_code}' ../missing.html ; curl --path-as-is
.../../etc/passwd
404
403

```

モード	処理の仕方	限界
<code>-m iterative</code>	一度にクライアント1つ	遅いクライアント1つが全員を止める
<code>-m fork</code>	接続ごとに子プロセス	プロセスの作成はスレッドの何倍もかかる（TD6・5で測定）
<code>-m thread</code>	接続ごとに新しいスレッド （ <code>pthread_detach</code> ）	何千もの接続 = 何千ものスレッド（メモリ、コンテキストスイッチ）
<code>-m pool</code>	固定の8スレッド + 有限のキュー （セマフォ + ミューテックス）	資源は限られる。遅いクライアント8つで飽和する（TD7・4）
<code>-m epoll</code>	1つのスレッドとイベントループ： もうデータのあるソケットだけを処理する	この簡単な版はまだブロッキングで書く。nginxやNode.jsは書き込みもブロックしない

テストはどのモードでも同時に40の要求（一度に8つ）を送り、40個の200応答と元と同一のバイナリを確かめる。最低限の安全対策：`..`を含むパスには**403**（クライアントは公開フォルダの外に出られない）、存在しないものには**404**。応答の途中でクライアントが切断しても死なないように、サーバはSIGPIPEを無視する。

⚠ 修正と補完

引き継いだ版は `201 Not found` と答え（正しいコードは404）、`..`を遮らず、`Content-Length`を送らず、部分的な書き込みを無視し、ファイルを開けたかを確かめず、課題文が求めるスレッド版とプール版がなかった。

アニメーション A8.2・サーバモデル →

TD7・4

● 新規・授業用マシンで測定

探究：カーネルはあなたのソケットで何をしているか

課題。(1) 接続の各段階で、各ソケットの状態を `getsockopt(TCP_INFO)` で読む。(2) `listen` の backlog が何を制限するかを調べる。(3) 遅いクライアントで Web サーバの5つのモードを測る。

(4) TCP と UDP を比べる。(5) 元のスライドの `htons` のバグを確かめ、`getaddrinfo` で名前を解決する。(6) ローカルソケットを作る。

1・TCP の状態 (`tcp_states.c`)

C tds/td7/tcp_states.c

```

    struct tcp_info ti;
    socklen_t len = sizeof ti;
    exit_if(getsockopt(s, IPPROTO_TCP, TCP_INFO, &ti, &len) == -1,
"getsockopt");
    return ti;
}

static const char *state(int s)
{
    if (s == -1)
        return "-";
    unsigned st = info(s).tcpi_state;
    return st < sizeof NAMES / sizeof NAMES[0] ? NAMES[st] : "?";
}

/* One line per step: the state of each socket. For a listening socket the
kernel reports the
    * accept queue in tcpi_unacked (connections waiting for accept) and
tcpi_sacked (backlog). */
static void show(const char *step, int l, int c, int a)
{
    struct tcp_info li = info(l);
    printf("%-26s | listener %s (queue %u/%u) | client %-11s | server %s\n",
        step, state(l), li.tcpi_unacked, li.tcpi_sacked, state(c),
state(a));
}

```

実際の出力

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -
| server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED
| server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED
| server ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2
| server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED
| server CLOSED
                           | the client's end moved to a TIME_WAIT entry of
the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

出力が示す 3 つの事実：接続は **accept** の前に確立している（ハンドシェイクはカーネルが行い、キューは 1/4 になる）。先に閉じた側は FIN_WAIT2 を通り、もう一方は **read** が 0 を返すまで CLOSE_WAIT に残る。そして閉じた後もカーネルは **TIME_WAIT** の項目（60 秒）を保ち、プログラムが終わっていても **ss** に見える。だからサーバは **SO_REUSEADDR** を使う。

2・backlog が制限するもの (backlog.c)

実際の出力

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5,
SYN_SENT 3
accept 5 (total 5), wait:   accept queue 3 | clients ESTABLISHED 8,
SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

listen(fd, 4) では Linux は **backlog + 1 = 5** 個のハンドシェイクを完了し、残りの SYN を捨てる。残りは SYN_SENT のまま SYN を再送する（1 秒、次に 2 秒、4 秒...）。サーバが **accept** するとキューが空き、再送が入る。backlog はサーバが処理するクライアントの数ではなく、**accept** を待てる完了済みの接続の数を制限する。

💡 興味のある人へ：SYN クッキー

実際の出力

```
$ nstat -n; ./backlog 2 8; nstat | grep -E
'Syncookies|ListenOverflows'
listen(2), 8 clients connect
0.3 s, no accept yet:          accept queue 3 | clients
ESTABLISHED 3, SYN_SENT 5
accept 3 (total 3), wait:      accept queue 3 | clients
ESTABLISHED 8, SYN_SENT 0
accept 3 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
accept 0 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
TcpExtSyncookiesSent 2
TcpExtListenOverflows 7
```

`listen(fd, 2)` ではもっと微妙なことが起きる：途中までのハンドシェイクのキューも満杯になると、カーネルは **SYN クッキー** で応答する（状態を持たない、SYN フラッドへの防御）。最後の ACK が届いたとき `accept` のキューが満杯なら、その ACK は失われる：クライアントは ESTABLISHED のつもりでも、サーバはそれを知らない。カーネルのカウンタ（`nstat`）がそれを裏付ける。だから実際のサーバの backlog は大きい（このマシンでは `net.core.somaxconn = 4096`）。

3・負荷の下での5つのモデル（`load.c`）



32 のクライアント：まず要求を送るのに1秒かかるものが1つ（次に8つ）。悪いネットワーク上のクライアントや *slowloris* 攻撃のようなもの。20 ms 後に速いクライアントが続く。各グループのレイテンシを `connect` から応答全体まで測る。

実際の出力

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 982 ms, max 983 ms
```

実際の出力

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 981 ms, max 981 ms
```

実際の出力

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 1 ms, max 2 ms
```

反復型：遅いクライアント1つで全員が約1秒待つ。8 スレッドのプール：遅いクライアント1つなら持ちこたえるが、8 つになると8 スレッドすべてが `read` でブロックし、同じことが起きる。`fork`、クライアントごとのスレッド、`epoll` は速いクライアントに数ミリ秒で応答する（`fork` と `thread` の出力は `tests/out/td7/` にある）。イベントループはそれを **スレッド1つ**でやってのける：

C tds/td7/http_server.c

```
/* ----- event loop: one thread watches every socket ----- */
static void event_loop(int s)
{
    int ep = epoll_create1(0);
    exit_if(ep == -1, "epoll_create1");
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = s }, ready[64];
    exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, s, &ev) == -1, "epoll_ctl");
    for (;;) {
        int n = epoll_wait(ep, ready, 64, -1); /* sleeps until some socket
is readable */
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "epoll_wait");
        for (int i = 0; i < n; i++) {
            int fd = ready[i].data.fd;
            if (fd == s) { /* a new connection: watch
it too */
                int c = accept(s, NULL, NULL);
                if (c == -1)
                    continue;
                ev.data.fd = c;
                exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, c, &ev) == -1,
"epoll_ctl");
            } else { /* the request has arrived:
read won't wait */
                exit_if(epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL) == -1,
"epoll_ctl");
                handle(fd);
                close(fd);
            }
        }
    }
}
```

アニメーション A8.2・サーバモデル →

4・TCP 対 UDP (udp.c)



実際の出力

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3
"two", recvfrom #3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8
received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

TCP は**バイトストリーム**：3 回の `write` が1回の `read` で届く（パイプと同じ）ので、各メッセージの終わりはアプリケーションのプロトコルが示す必要がある。UDP は各データグラムの**区切り**を保つが、受け手を待たない：バッファが満杯だと、カーネルは同じマシンの中でさえ誰にも知らせずにデータグラムを捨て、`/proc/net/snmp` に記録するだけ。

5・アドレスとポート（`addr.c`）・6・ローカルソケット（`local.c`）



実際の出力

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110,
sockaddr_storage 128
```

実際の出力

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path
holds 108 bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

スライド 187 の例は `htons` なしで `sin_port = 7000` としていた：リトルエンディアンのプロセッサでは 2 バイトが逆向きに読まれ、カーネルはポート **22555** を開く。（同じ

スライドの) `gethostbyname` は IPv4 しか知らず、スレッドセーフでなく、2008 年に POSIX から外れた：`getaddrinfo` は同じ呼び出しで IPv4 と IPv6 を解決する。`AF_UNIX` ソケットは 127.0.0.1 を使わない（それはループバックインターフェース上の `AF_INET`）：アドレスは **ソケット型のファイル**で、カーネルは誰が接続したかをサーバに教える（`SO_PEERCREC`）。`systemd`、`D-Bus`、`Docker` はこれで権限を決める。

アニメーション A8.1・接続のライフサイクル →

チェックリスト

生成AIチャレンジ・決してブロックしない`epoll`サーバ



目標。 `http\_server.c` の `-m epoll` モード（まだブロッキングで読み書きする）を、非ブロッキングのソケットと部分的な書き込みを扱う完全なイベントループにし、何千もの接続でクライアントごとのスレッドのモードと比べて測る。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

5 つのモードを持つ C の HTTP サーバがあります。epoll モードはスレッド 1 つですが、`handle()` がブロックする read と write を行います [http_server.c と load.c を貼り付け]。次を手伝ってください：(1) `accept4(..., SOCK\_NONBLOCK)` を使い、接続ごとに状態を持つ（要求を \r\n\r\n まで読み、オフセット付きで応答を書く）。`write` が EAGAIN を返したときだけ EPOLLOUT を登録する。(2) `load.c` を拡張して、とてもゆっくり送る 2000 の接続を開く。(3) 速いクライアントのレイテンシ、スレッド数（`ps -o nlwp`）、メモリ（`ps -o rss`）を測って、epoll モードと `-m thread` を比べる。C10K 問題と、各モードを制限する OS の資源は何かを説明してください。

結果の確かめ方

- 大きなファイルをととてもゆっくり読むクライアント（`curl --limit-rate 10k`）が他を遅らせない：`./load` がそれを示す。
- `tests/run.sh td7` は引き続き通り、バイナリファイルは同一のまま届く（`cmp`）。
- 表はあなたの測定から作り、どのマシンで測ったかを書く。
- 同時接続数を上げたときに見える失敗の少なくとも1つを`ulimit -n`で説明できる。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。